Last Name: First Name:

Matriculation Number:

Exam
Artificial Intelligence 1

March 31, 2026

Please ignore the QR codes; do not write on them, they are for grading support

	To be used for grading, do not write here														
prob.	1.1	1.2	2.1	3.1	3.2	3.3	4.1	5.1	6.1	6.2	6.3	7.1	8.1	Sum	grade
total	4	4	8	12	9	4	6	7	6	7	7	6	10	90	
reached															

Old exams are great practice resources, but since exam styles evolve, recent exams are better models, and any course topic even if not asked before can still appear in upcoming exams. Note that these exams are for learning support only, are not exhaustive or guaranteed correct, and may not reflect the actual quiz or exam questions.

The exams here are not necessarily identical to the old exams. For technical reasons, the points may change and some sub-problems that were elided in teh exam may appear here.

In the courses Artificial Intelligence I/II and SMAI we award bonus points for the first student who reports a factual error in an old exam. (Please report spelling/formatting errors as well.)

1 General Aspects

Problem 1.1 (EU AI Act)

1. Name the 4 risk categories defined in the EU AI Act and give an example for each. 4 Points

---

Solution: The categories are

- minimal risk: e.g., games, spam filters
- limited risk: e.g., image, video generation
- high risk: significant threats to health, safety, or the fundamental rights of persons such as uses in health, education, recruitment, critical infrastructure management, law enforcement, judiciary
- unacceptable risk: e.g., manipulate human behavior, real-time remote biometric identification in public spaces, social scoring (ranking individuals based on their personal characteristics, socio-economic status, or behavior)

Problem 1.2 (Environments of Rational Agents)

1. Briefly explain the difference between dynamic and static environments. Give an example for the latter. 2 Points

---

Solution: An environment is dynamic if it can change without the agent taking an action, otherwise static. An example is an agent playing chess.

---

2. Briefly explain the difference between fully and partially observable environments. Give an example for the latter. 2 Points

---

Solution: An environment is fully observable if the agent knows the complete state of the environment, otherwise partially observable. An example is an agent playing a game with hidden information, such as poker.

---

2 Prolog

Problem 2.1 (Interpreting Programs)
Consider the following Prolog program (where `isin(U,L)` checks if `U` is in the list `L`):

```
isin(U,[U|_]).
isin(U,[_|R]) :- isin(U,R).

isProp(pv(P),PS) :- isin(P,PS).
isProp(neg(A),PS) :- isProp(A,PS).
isProp(conj(A,B),PS) :- isProp(A,PS), isProp(B,PS).

evaluate(pv(P), VS, X) :- isin((P,X),VS).
evaluate(neg(A), VS, 1) :- evaluate(A,VS,0).
evaluate(neg(A), VS, 0) :- evaluate(A,VS,1).
evaluate(conj(A,B), VS, 1) :- evaluate(A,VS,1), evaluate(B,VS,1).
```

```
evaluate(conj(A,_), VS, 0) :- evaluate(A,VS,0).
evaluate(conj(_,B), VS, 0) :- evaluate(B,VS,0).
```

1. Which operation related to the course does the predicate `isProp(A,PS)` implement? 3 Points

   Solution: It checks if `A` is a propositional formula using propositional variables from `PS`.

2. Which result(s) does Prolog return for the query `evaluate(conj(pv(p),neg(pv(q))), [(p,1),(q,0)], X)`? 2 Points

   Solution: One result, `X=1`.

3. Assume we replace the last 3 lines with 3 Points

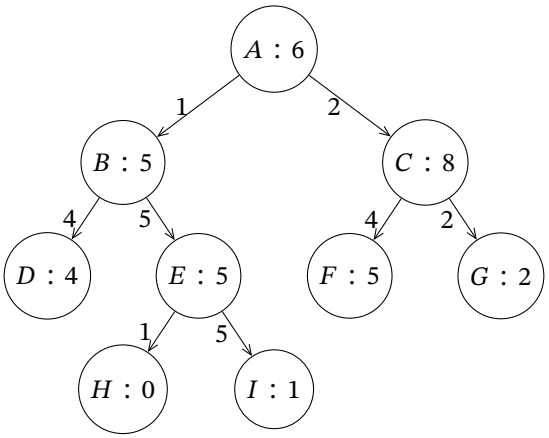
   ```
   evaluate(conj(A,B), VS, X) :- evaluate(A,VS,1), evaluate(B,VS,X).
   evaluate(conj(A,_), VS, 0) :- evaluate(A,VS,0).
   ```

   What effects (if any) does this have on the result and the efficiency of the program?

   Solution: It has no effect on the result. Execution is faster for certain conjunctions (namely those where the first argument is true and the second false).

3 Search

Problem 3.1 (Search Algorithms)
Consider the tree below. Every edge is labeled with its cost.
Each node is labeled with $n : h(n)$ where n is its name and $h(n)$ its value for a heuristic h. H is the only goal state.



1. h is not consistent. Which two edges can be used as examples to demonstrate that? 2 Points

   Solution: $C \to G$ and $E \to H$ (because h decreases by more than the edge cost).

2. h is not admissible. What concrete change can you make to h to obtain an admissible heuristic? 2 Points

Solution: $h(E)$ must be ≤ 1. ($h(n)$ must be less than $h^*(n)$, i.e., the cost to the goal. Only A, B, E, H matter because h^* is ∞ for the other nodes. h^* for $A/B/E/H$ is $7/6/1/0$. So only $h(E)$ must change.)

Assume you have already expanded the root node A. In each problem below, list the **next** 4 nodes that will be expanded using the respective search algorithm and (if needed) heuristic h.
If there is a tie, break it using alphabetical order.

3. depth-first search 1 Points

Solution: B, D, E, H

4. breadth-first search 1 Points

Solution: B, C, D, E

5. uniform-cost search 2 Points

Solution: B, C, G, D

6. greedy search 2 Points

Solution: B, D, E, H

7. A^*-search 2 Points

Solution: B, D, C, G

Problem 3.2 (Abstract Search Problems)
Consider the following search problem:

- set S of states: $\{a, b, c, d, e, f, g, h, i\}$
- set A of actions: $\{x, y, z\}$
- transition model T: as given by the table
- initial states I: the states marked by $\rightarrow$
- goal states G: $s \in G$ iff s is marked by !

	x	y	z
$\rightarrow a$	$\{d\}$	$\{\}$	$\{\}$
b	$\{e\}$	$\{\}$	$\{\}$
c	$\{f\}$	$\{\}$	$\{\}$
d	$\{g\}$	$\{\}$	$\{\}$
$!\, e$	$\{h\}$	$\{a\}$	$\{\}$
f	$\{i\}$	$\{b\}$	$\{a\}$
g	$\{\}$	$\{c\}$	$\{b\}$
h	$\{\}$	$\{d\}$	$\{c\}$
i	$\{\}$	$\{e\}$	$\{d\}$

1. This search problem is determinisitic. How can we tell? 1 Points

   Solution: The transition model always returns a set containing at most one element.

2. Give the state(s) that can be reached by applying the action sequence x, x, y in an initial state. 2 Points

   Solution: The possible states are c.

3. Give all 3 solutions whose length is at most 8. 3 Points

   Solution: The solutions are

   - $a \xrightarrow{x} d \xrightarrow{x} g \xrightarrow{z} b \xrightarrow{x} e$
   - $a \xrightarrow{x} d \xrightarrow{x} g \xrightarrow{y} c \xrightarrow{x} f \xrightarrow{x} i \xrightarrow{y} e$
   - $a \xrightarrow{x} d \xrightarrow{x} g \xrightarrow{y} c \xrightarrow{x} f \xrightarrow{y} b \xrightarrow{x} e$

   The list of actions is the solution. The states are only included above for illustration.

4. In this search problem, we can alternatively search backwards by starting in a goal state and applying the transition relation in reverse until we reach an initial state. Briefly give two reasons why this is not feasible for every search problem. 3 Points

   Solution: There are multiple reasons:

   - If the state space is large, we might not have a good way to determine the goal states a priori (e.g., searching for gold).
   - There may be a lot more goal states than initial states, making the search inefficient (e.g., playing chess).
   - The transition relation might be given in a way that is not invertible or expensive to invert (e.g., a moving robot).
   - The branching factor when searching backwards might be huge, e.g., if there are many states that can lead to a goal state but are unreachable from the initial state (e.g., becoming a millionaire).

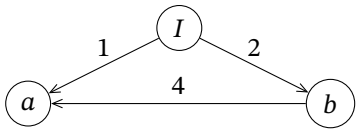
   Not good reasons are that it does not work for non-deterministic or partially observable problems or that there may many states or we may get stuck in loops — those issues also come up when searching forward, and some problems might be easier when done backwards. Also, note that searching backwards is not the same as backtracking.

Problem 3.3 (Redundant Expansion)
The search algorithms presented in the lecture store the nodes that are currently in the fringe, but they do not remember which nodes have already been expanded before. Therefore they might redundantly expand the same node twice.

1. Assume you have optimized BFS in such a way that a node is **not** inserted into the fringe if it already is in the fringe. 4 Points

Give an example of an **acyclic** graph in which it still happens that BFS expands the same node twice.
Give the list of nodes in the order of expansion until a node is expanded for the second time.

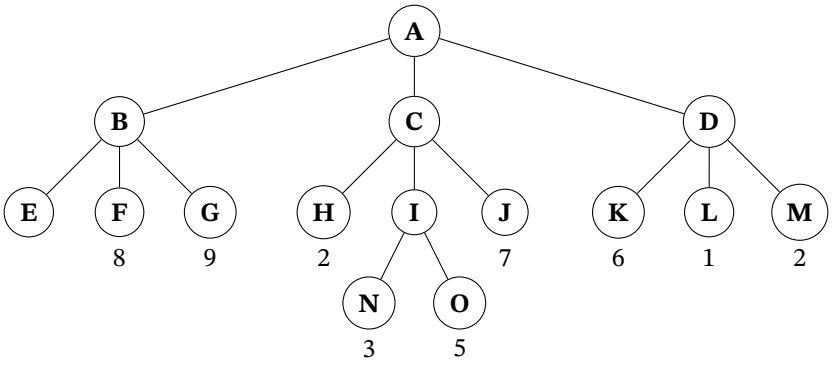
Solution: Many possibilities. The key is to have two paths from the initial node to some node a in such a way that (i) a is found and added to the fringe, (ii) a is expanded and thus removed from the fringe, and then (iii) a is found again.
The easiest such graph is the one below (with the understanding that the children of I are ordered alphabetically). The expanded nodes are I, a, b, a.



---

4 Adversarial Search

Problem 4.1 (Minimax)
Consider the following minimax game tree for the **maximizing** player's turn. The values at the leaves are the static evaluation function values of those states; some of those values are currently missing.



1. Label the nodes I and C with their minimax value. 2 Points

---

Solution: I: 5, C: 2

---

2. If possible, label the node **E** with an evaluation function value that results in the player definitely (i.e., no matter how ties are broken) choosing move **C**. 2 Points
Otherwise, argue why that is impossible.

Solution: Any label < 2.

---

3. Now assume **E** is labeled with 5, and we use $\alpha\beta$-pruning. We expand child nodes in alphabetical order. Which nodes would be pruned? 2 Points

---

Solution: I, N, O, J, M (or I,J,M)

5 Constraint Satisfaction

Problem 5.1 (Kinds and Solutions)
Consider the following CSP:

- variables: a, b, c
- domains: $D_a = D_b = D_c = \mathbb{N}$
- constraints: $a + b < c, \quad a + c = 5, \quad 2 \cdot a + b^2 < c^7 - b^5$

1. In the following table, check for each statement whether it is true or false. (You will not get penalized for wrong answers.) 3 Points

The problem is …	true	false
finite		
discrete		
continuous		
binary		
higher-order		

Solution: It is discrete and higher-order but not the others. 3 correct answers (better than guessing) were needed to score points. Wrong answers were counted the same way as non-answers.

2. Which constraint should you start with when searching for a solution, and why? 2 Points

Solution: The second one. Because it is an equality, arc-consistency eliminates more domain values, in this case all but finitely many possible values for a and c.

3. Give all 2 solutions that satisfy $a = b$. 2 Points

Solution: $(a, b, c) \in \{(0, 0, 5), (1, 1, 4)\}$ (These are the only assignments that satisfy $a = b$ and the first two constraints. Since the question confirmed that there are exactly 2 solutions, the third constraint can be ignored.)

6 Logic

Problem 6.1 (Propositional Logic)
Consider the set $\{P, Q\}$ of propositional variables.

1. Give all assignments φ that **satisfy** the formula A given by $(P \wedge Q) \Rightarrow (Q \wedge P)$. 2 Points

Solution: All assignments, i.e., $\varphi(P), \varphi(Q) \in \{\mathsf{T}, \mathsf{F}\}$. Assignments that use other truth values (e.g., 0 and 1) or that are given as a table are also accepted.

2. Give an assignment ψ that **falsifies** the formula B given by $(P \vee Q) \Rightarrow (P \wedge Q)$. 2 Points

Solution: Any assignment with $\psi(P) \neq \psi(Q)$, i.e., $(\psi(P), \psi(Q)) \in \{(\mathsf{T}, \mathsf{F}), (\mathsf{F}, \mathsf{T})\}$. Assignments that use other truth values (e.g., 0 and 1) or that are given as a table are also accepted.

3. For each of the following formulas, check if they are valid, satisfiable but not valid, and/or unsatisfiable. (You will not be penalized for wrong answers.) 2 Points

formula	valid	satisfiable but not valid	unsatisfiable
$P \Rightarrow \neg P$			
$(P \Rightarrow Q) \Rightarrow (\neg Q \Rightarrow \neg P)$			
$(T \Rightarrow P) \vee (P \Rightarrow F)$			

Solution: They are, respectively, satisfiable but not valid, valid, valid. Wrong answers were counted in the same way as non-answers. Multiple checkmarks in one row counted as a wrong answer.

Problem 6.2 (First-Order Models)
Consider the syntax of integers using the signature with a

- nullary function symbol `zero`,
- binary function symbol `plus`,
- binary predicate symbol `less`.

1. Give the model that assigns the usual interpretations in the **integers**. 3 Points

Solution: The model is $\langle \mathbb{Z}, I \rangle$ (i.e., the domain/universe is the set $\mathbb{Z}$ of integers) where the interpretations $I(-)$ of the symbols are:

- $I(\texttt{zero}) = 0$
- $I(\texttt{plus})(u, v) = u + v$
- $I(\texttt{less}) \ni \langle u, v \rangle$ iff $u < v$

for $u, v \in \mathbb{Z}$

2. Briefly explain whether it is possible to give a **finite** model for the above signature. 1 Points

Solution: Yes. Any finite (non-empty) set can be used as the universe — we can always give some some interpretation for the function and predicate symbols.

3. Now give a model for the above signature that **satisfies** the formula $\forall x.\texttt{less}(\texttt{plus}(x,x),\texttt{zero})$. 3 Points

Solution: Lots of options. A trivial choice is $M = \langle\{0\}, I\rangle$. Because the universe is a singleton set, the interpretation of function symbols is already determined:
- $I(\texttt{zero})(0) = 0$
- $I(\texttt{plus})(0,0) = 0$

For predicate symbols, only 2 options exists. The formula is satisfied if we make the interpretation of the predicate true:
- $I(\texttt{less}) = \{(0,0)\}$

Problem 6.3 (Logical Systems)
Let $V = \{X, Y, Z\}$ be a fixed set of names.
Consider the logical system $\langle \mathcal{L}, \mathcal{M}, \models \rangle$ given by
- $\mathcal{L}$ is the set of formulas of the form
  - P for $P \in V$
  - $\triangle$
  - $A \oplus B$ for $A, B \in \mathcal{L}$
- The models $M \in \mathcal{M}$ are the functions $M : V \to \{\mathsf{F}, \mathsf{T}\}$.
- $\models$ is defined by
  - $M \models P$ iff $M(P) = \mathsf{T}$
  - $M \models \triangle$ (always holds)
  - $M \models A \oplus B$ iff either $M \models A$ or $M \models B$ (but not both)

1. Consider the model M given by $M(P) = \mathsf{T}$ for all $P \in V$. Give a formula A such that $M \not\models A$ (i.e., it is not the case that $M \models A$). 3 Points

Solution: Note that $\triangle$ behaves like T, and $\oplus$ behaves like XOR.
So $A = X \oplus X$ or $A = X \oplus \triangle$ works. More generally, any formula that uses an even number of arguments.

2. Give a valid formula. 2 Points

Solution: $\triangle$ works. More generally, any formula works if it uses an odd number of $\triangle$ and an even number of every variable, e.g., $(X \oplus X) \oplus \triangle$.

3. Give the appropriate tableau rule for $(A \oplus B)^{\mathsf{T}}$. 2 Points

Solution:

$(A \oplus B)^{\mathsf{T}}$	
A^{T}	A^{F}
B^{F}	B^{T}

7 Knowledge Representation

Problem 7.1 (ALC)
Consider the following ALC signature

- concept symbols: p (for person), a (for animal)
- role symbols: o (for owns)

1. Give a concept that represents the set of people that own animals. 1 Points

   Solution: $p \sqcap \exists o.a$

2. Give an axiom that states that people cannot be owned. 2 Points

   Solution: $(\exists o.p) \sqsubseteq \bot$

3. Give an ABox that represents a person Alice owning an animal Bubbles. 1 Points

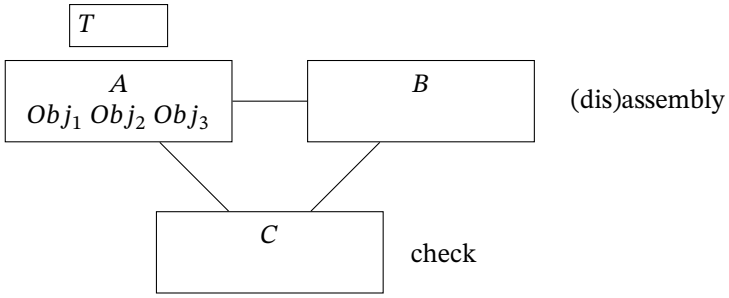
   Solution: $Alice : p, Bubbles : a, Alice\ o\ Bubbles$
   Drawing the ABox as a semantic network was also accepted.

4. Calculate the translation to first-order logic of the concept $\forall o.\exists o.a$. 2 Points

   Solution: $\forall x.o(u,x) \Rightarrow (\exists y.o(x,y) \wedge a(y))$ with free variable u

8 Planning

Problem 8.1 (STRIPS)
Consider a machine that processes the objects in $Obj = \{Obj_1, Obj_2, Obj_3\}$, which can be at location A, B, or C. Currently all objects are at location A and **unchecked** and **assembled**. Eventually all objects are needed in location A and **checked** and **assembled**.
At location B, objects can be assembled or disassembled. At location C, disassembled objects can be checked.
A transporter T is available (currently at location A) that can move **one** object at a time from one location to any other location.



We formalize this problem as a STRIPS task where the **facts** are

- $\texttt{at}(l, o)$ for $l \in \{A, B, C\}$ and $o \in Obj \cup \{T\}$
- $\texttt{isCh}(o)$ and $\text{isAss}(o)$ and $\texttt{isDis}(o)$ for $o \in Obj$

and the **actions** are given by table below for any $l, m \in \{A, B, C\}$ and $o \in Obj$

action	precondition	add list	delete list
$\text{move}(l, m, o)$	$\texttt{at}(l, o)$	$\texttt{at}(m, o)$	$\texttt{at}(l, o)$
$\text{assemble}(o)$	$\texttt{at}(B, o)$	$\text{isAss}(o)$	$\texttt{isDis}(o)$
$\texttt{disassemble}(o)$	$\texttt{at}(B, o)$	$\texttt{isDis}(o)$	$\text{isAss}(o)$
$\texttt{check}(o)$			

1. Complete the definition of the action $\texttt{check}(o)$. 2 Points

   Solution:
   - precondition: $\texttt{at}(C, o)$, $\texttt{isDis}(o)$
   - add list: $\texttt{isCh}(o)$
   - delete list: nothing

2. Give the initial state I and the goal state G. 2 Points

   Solution: Initial state: $\texttt{at}(A, o)$ and $\text{isAss}(o)$ for all $o \in Obj$, and $\texttt{at}(A, T)$, goal: $\texttt{at}(A, o)$, $\texttt{isCh}(o)$ and $\text{isAss}(o)$ for all $o \in Obj$

3. Give the value $h^*(I)$. 2 Points

   Solution: 21. An optimal plan requires 7 actions per object: $\text{move}(A, B, o)$, $\texttt{disassemble}(o)$, $\text{move}(B, C, o)$, $\texttt{check}(o)$, $\text{move}(C, B, o)$, $\text{assemble}(o)$, $\text{move}(B, A, o)$

4. Give the value $h^+(I)$. 2 Points

   Solution: 12. With the delete heuristic, objects do not have to re-assembled or moved back. So only 4 actions are needed per object.

5. Prove or refute that the following heuristic is admissible: $h(s)$ is 5 times the number of unchecked objects in state s. 2 Points

   Solution: It is not admissible. A counterexample is the state s with T and one disassembled, unchecked object at location C and all other objects checked and assembled at A. Then $h^*(s) = 4 < 5 = h(s)$ in violation of the admissibility condition.